

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 3 PERTEMUAN 1

SEMESTER III



BKPM ACARA 5

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Tanpa routing, URL aplikasi web PHP native umumnya mengikuti lokasi berkas fisiknya, misalnya, `http://localhost/siakademik/app/Controllers/MahasiswaController.php?action=index`. URL semacam ini tidak rapi, membocorkan struktur folder aplikasi, dan sulit diingat pengguna. Dengan menerapkan routing sederhana yang dipadukan dengan `.htaccess` (URL rewriting), URL dapat dibuat bersih, misalnya `http://localhost/mahasiswa` atau `http://localhost/mahasiswa/5`.

Routing adalah proses memetakan URL yang diminta client ke kode program yang tepat, biasanya berupa method pada sebuah Controller. Seluruh request yang masuk diarahkan terlebih dahulu ke satu titik masuk (front controller), yaitu `public/index.php`, yang kemudian membaca daftar rute pada `routes/web.php` untuk menentukan Controller dan method mana yang harus dijalankan.

2. Tujuan

1. Membuat file `public/.htaccess` agar seluruh request non-file diarahkan ke `index.php`.
2. Membuat `routes/web.php` sebagai daftar pemetaan URL ke Controller.
3. Membuat `public/index.php` sebagai front controller yang membaca URL, mencocokkan rute, lalu memanggil Controller yang sesuai.
4. Membuat `HomeController` dan `MahasiswaController` sederhana (masih berupa echo).
5. Menguji URL `/`, `/mahasiswa`, `/mahasiswa/create`, serta URL yang tidak terdaftar (harus menampilkan 404).

3. Dasar Teori Singkat

Array Routing. Daftar rute disimpan sebagai array PHP bertingkat, dengan format `$routes[METHOD][path] = [NamaController, method]`. Array ini memisahkan rute berdasarkan HTTP method (GET, POST, dan seterusnya), sehingga path yang sama dapat memiliki perilaku berbeda tergantung method yang digunakan.

.htaccess dan URL Rewriting. Berkas `.htaccess` pada folder `public` memberi tahu Apache agar setiap request yang tidak menunjuk ke file atau folder yang benar-benar ada diteruskan ke

index.php, dengan query string tetap dipertahankan melalui flag QSA (Query String Append) dan L (Last rule, berhenti setelah aturan ini cocok).

Front Controller (index.php). Seluruh request masuk melalui satu berkas ini. Front controller bertugas mengambil URL yang diminta, mengambil HTTP method, mencari rute yang cocok pada array routes, memanggil Controller@method yang sesuai, dan menampilkan halaman 404 apabila rute tidak ditemukan.

Superglobal Penting. \$_GET menyimpan data dari query string, \$_POST menyimpan data dari body form POST, dan \$_SERVER menyimpan informasi server dan request seperti REQUEST_URI dan REQUEST_METHOD yang digunakan front controller untuk menentukan rute.

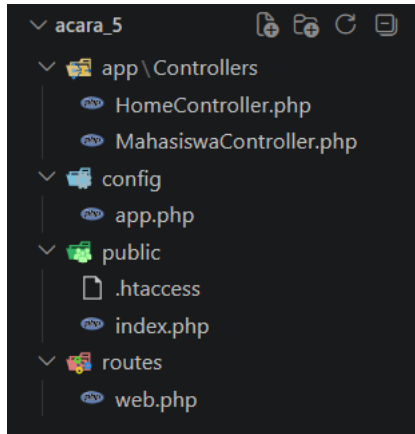
4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon (dengan modul mod_rewrite aktif)
- PHP
- Modul Praktikum (BKPM Acara 5)

HASIL PRAKTIK

1. Struktur Folder Project

Sesuai langkah kerja pada BKPM, dibuat struktur folder berikut di dalam htdocs:



2. Membuat public/.htaccess

File ini mengarahkan seluruh request yang bukan file/folder nyata ke index.php, sehingga URL bersih seperti /mahasiswa dapat diproses oleh front controller.

```
public > .htaccess
1 # public/.htaccess
2 RewriteEngine On
3 RewriteCond %{REQUEST_FILENAME} !-f
4 RewriteCond %{REQUEST_FILENAME} !-d
5 RewriteRule ^ index.php [QSA,L]
```

3. Membuat routes/web.php

Daftar rute GET dan POST didefinisikan dalam satu array. Rute /mahasiswa/{id} sengaja ditambahkan sebagai rute dinamis untuk kebutuhan Tugas Mandiri, ditandai dengan tanda kurung kurawal pada segmen id.

```
routes > web.php
1 <?php
2 $routes = [
3     'GET' => [
4         '/' => ['HomeController', 'index'],
5         '/mahasiswa' => ['MahasiswaController', 'index'],
6         '/mahasiswa/create' => ['MahasiswaController', 'create'],
7         '/mahasiswa/{id}' => ['MahasiswaController', 'show'],
8     ],
9     'POST' => [
10         '/mahasiswa' => ['MahasiswaController', 'store'],
11     ],
12 ];
```

4. Membuat HomeController dan MahasiswaController

Kedua Controller masih sederhana (langkah 4 pada prosedur kerja), berisi echo untuk menandai bahwa routing berhasil memanggil method yang tepat. MahasiswaController juga menyimpan data dummy agar method show(\$id) pada Tugas Mandiri dapat diuji tanpa database.

```
app > Controllers > HomeController.php
1 <?php
2 namespace App\Controllers;
3
4 class HomeController
5 {
6     public function index(): void
7     {
8         echo "<h1>Selamat datang</h1>";
9         echo "<p>Ini adalah halaman utama (Home). Diakses melalui routing bersih tanpa .php.</p>";
10    }
11 }
```

```
1 <?php
2 namespace App\Controllers;
3
4 class MahasiswaController
5 {
6     private array $mahasiswa = [
7         1 => ['nim' => '2401001', 'nama' => 'Budi Santoso'],
8         2 => ['nim' => '2401015', 'nama' => 'Siti Aminah'],
9         3 => ['nim' => '2302032', 'nama' => 'Andi Wijaya'],
10    ];
11
12    public function index(): void
13    {
14        echo "<h1>Daftar Mahasiswa</h1>";
15        echo "<p>Method: MahasiswaController::index() dipanggil melalui GET /mahasiswa</p>";
16        echo "<ul>";
17        foreach ($this->mahasiswa as $id => $mhs) {
18            echo "<li>#{$id} - {$mhs['nim']} - {$mhs['nama']} " .
19                "<a href='\"/mahasiswa/{$id}\"\">detail</a></li>";
20        }
21        echo "</ul>";
22    }
23
24    public function create(): void
25    {
26        echo "<h1>Form Tambah Mahasiswa</h1>";
27        echo "<p>Method: MahasiswaController::create() dipanggil melalui GET /mahasiswa/create</p>";
28    }
29
30    public function store(): void
31    {
32        echo "<h1>Simpan Mahasiswa</h1>";
33        echo "<p>Method: MahasiswaController::store() dipanggil melalui POST /mahasiswa</p>";
34    }
35
36    public function show(string $id): void
37    {
38        echo "<h1>Detail Mahasiswa</h1>";
39        echo "<p>Method: MahasiswaController::show({$id}) dipanggil melalui GET /mahasiswa/{$id}</p>";
40
41        if (isset($this->mahasiswa[$id])) {
42            $mhs = $this->mahasiswa[$id];
43            echo "<p>NIM: {$mhs['nim']}<br>Nama: {$mhs['nama']}</p>";
44        } else {
45            echo "<p>Data dengan id {$id} tidak ditemukan.</p>";
46        }
47        echo "<p><a href='\"/mahasiswa\"\">Kembali ke daftar</a></p>";
48    }
49 }
```

5. Membuat public/index.php (Front Controller)

Front controller membaca URL dan method, lalu mencoba dua tahap pencocokan: (1) rute statis persis sama, dan (2) rute dinamis berpola {id} menggunakan regex. Base path project dihitung otomatis dari lokasi index.php (melalui SCRIPT_NAME), sehingga project tetap berjalan baik diakses langsung maupun dari dalam subfolder htdocs.

```
public > index.php
1  <?php
2
3  require_once __DIR__ . '/../config/app.php';
4  require_once __DIR__ . '/../routes/web.php';
5
6  $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
7
8  $base = rtrim(str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME'])), '/');
9  if ($base !== '' && str_starts_with($uri, $base)) {
10     $uri = substr($uri, strlen($base));
11 }
12 $uri = $uri === '' ? '/' : $uri;
13
14 $method = $_SERVER['REQUEST_METHOD'];
15
16 if (isset($routes[$method][$uri])) {
17     [$controllerName, $action] = $routes[$method][$uri];
18     dispatch($controllerName, $action);
19     exit;
20 }
21
22 foreach ($routes[$method] ?? [] as $pattern => $target) {
23     if (!str_contains($pattern, '{') {
24         continue;
25     }
26     // Ubah "/mahasiswa/{id}" menjadi regex "#^/mahasiswa/([^\s/]+)$#"
27     $regex = '#^' . preg_replace('/\{[a-zA-Z_]+\}/', '([^\s/]+)', $pattern) . '$#';
28     if (preg_match($regex, $uri, $matches)) {
29         array_shift($matches); // buang full match, sisakan parameter
30         [$controllerName, $action] = $target;
31         dispatch($controllerName, $action, $matches);
32         exit;
33     }
34 }
35
36 http_response_code(404);
37 echo "404 - Halaman tidak ditemukan";
38 exit;
39
40 function dispatch(string $controllerName, string $action, array $params = []): void
41 {
42     $controllerClass = "App\\Controllers\\" . $controllerName;
43     require_once __DIR__ . '/../app/Controllers/' . $controllerName . ".php";
44     $controller = new $controllerClass();
45     call_user_func_array([$controller, $action], $params);
46 }
```

6. Hasil Pengujian

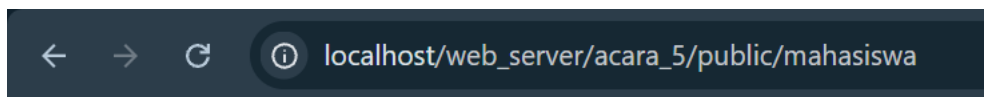
Berikut hasil pengujian tiap endpoint melalui browser setelah project diaktifkan pada Apache (XAMPP).

a) GET / → HomeController::index()

Selamat datang

Ini adalah halaman utama (Home). Diakses melalui routing bersih tanpa .php.

b) GET /mahasiswa → MahasiswaController::index()

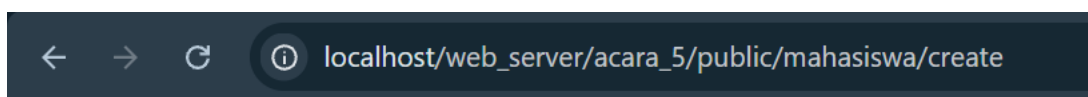


Daftar Mahasiswa

Method: MahasiswaController::index() dipanggil melalui GET /mahasiswa

- #1 - 2401001 - Budi Santoso ([detail](#))
- #2 - 2401015 - Siti Aminah ([detail](#))
- #3 - 2302032 - Andi Wijaya ([detail](#))

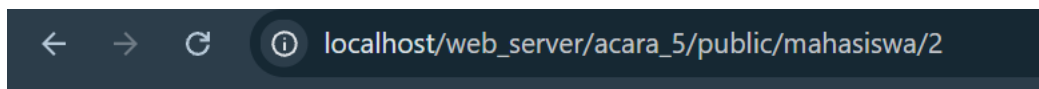
c) GET /mahasiswa/create → MahasiswaController::create()



Form Tambah Mahasiswa

Method: MahasiswaController::create() dipanggil melalui GET /mahasiswa/create

d) GET /mahasiswa/2 (Tugas Mandiri: parameter URL) → MahasiswaController::show(\$id)



Detail Mahasiswa

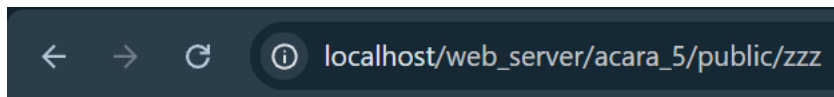
Method: MahasiswaController::show(\$id) dipanggil melalui GET /mahasiswa/2

NIM: 2401015

Nama: Siti Aminah

[Kembali ke daftar](#)

e) GET /tidak-ada (rute tidak terdaftar) → 404



404 - Halaman tidak ditemukan

7. Analisis dan Pembahasan

Pengujian menunjukkan bahwa seluruh request, baik ke / maupun ke /mahasiswa dan turunannya, benar-benar diproses oleh satu berkas public/index.php, bukan oleh berkas Controller secara langsung. Hal ini dibuktikan dengan tidak adanya ekstensi .php pada URL yang diakses, sesuai tujuan .htaccess pada langkah 1.

Untuk Tugas Mandiri, dukungan parameter URL diimplementasikan dengan menandai rute dinamis menggunakan pola {id} pada routes/web.php, kemudian front controller mengubah pola tersebut menjadi regular expression ([^/]+) untuk menangkap nilai id dari URL. Ketika /mahasiswa/2 diakses, front controller mencocokkan pola /mahasiswa/{id}, mengekstrak nilai "2", lalu memanggil MahasiswaController::show("2"). Pendekatan ini dipilih dibandingkan explode('/', \$url) sederhana karena lebih mudah dikembangkan apabila jumlah rute dinamis bertambah, tanpa harus menulis pengecekan manual untuk setiap pola URL.

Ketika URL yang diminta tidak cocok dengan rute statis maupun rute dinamis manapun, front controller mengembalikan status HTTP 404 beserta pesan "404 - Halaman tidak ditemukan", sesuai perilaku yang diharapkan pada langkah pengujian BKPM.

KESIMPULAN

1. Routing memetakan URL request ke Controller dan method yang tepat melalui array `$routes`, dipisahkan berdasarkan HTTP method (GET/POST).
2. `.htaccess` dengan `RewriteRule` mengarahkan seluruh request non-file ke `public/index.php`, sehingga URL aplikasi menjadi bersih tanpa ekstensi `.php` dan tanpa membocorkan struktur folder.
3. `public/index.php` sebagai front controller adalah satu-satunya titik masuk aplikasi: mengambil URL dan method, mencocokkan rute, lalu memanggil `Controller@method` yang sesuai.
4. Dukungan parameter URL (Tugas Mandiri) diterapkan dengan pola `{id}` pada definisi rute yang dikonversi menjadi regex oleh front controller, memungkinkan satu rute menangani banyak nilai id secara dinamis, seperti pada `/mahasiswa/2`.
5. URL yang tidak cocok dengan rute manapun ditangani dengan mengirim status HTTP 404, sehingga pengguna maupun sistem lain dapat mengetahui bahwa halaman yang diminta tidak tersedia.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.